Antiforensics

What is Antiforensics?

Anti-forensics is a set of techniques used by attackers to obstruct, delay, or evade digital forensic investigations by deleting, hiding, or manipulating digital evidence. It involves covering digital footprints through methods like data wiping, encryption, and log manipulation to make it difficult for investigators to reconstruct events.

Key aspects and techniques include:

- Data Wiping/Overwriting: Permanently deleting files and overwriting data to make recovery impossible.
- Encryption: Protecting sensitive data from being accessed or read by investigators.
- Steganography: Concealing malicious data within other files.
- Artifact Wiping/Log Cleaning: Deleting system logs or manipulating file metadata to erase traces of activity.
- Obfuscation: Hiding data in plain sight or using ambiguous code to confuse analysts.
- Data Hiding: Using hidden partitions or creating "bad blocks" to conceal data from the operating system

Antiforensics refers to techniques used to:

- Hide malicious activity
- Destroy digital evidence
- Make forensic investigation difficult

Attackers try to:

- Remove logs
- Modify timestamps
- Hide data inside files
- Encrypt or obfuscate malware

What Attackers Try to Do

Attackers attempt to:

- Delete system logs
- Clear authentication logs
- Remove command history
- Disable logging services

Purpose:

- Hide login attempts
- Hide privilege escalation
- Hide malware execution traces

Top Anti-Forensics Techniques

1. Steganography: This technique is used to hide data within another file, image, audio, or video so that the existence of the concealed data is difficult to detect. Unlike encryption, which only conceals the message's content, steganography hides the fact that a message exists at all.

Common Methods

- **Image Steganography:** Hide data within the least significant bits of pixel values in image files. This method slightly alters an image's color values but is invisible to the human eye.

- **Audio and Video Steganography:** Similar techniques can be applied to audio or video files, embedding data of the audio sample values or frames in the least significant bits.

- **Text Steganography:** Modify text documents by adjusting character formatting, adding invisible characters, or using context-free grammar.

Tools Used: Popular tools include **Steghide**, **StegoSuite**, **OpenStego,** and **SilentEye**.

2. Data Obfuscation: This technique makes data intentionally challenging to read or analyze, not by hiding it but by making it confusing and complex.

Common Methods

- **Encryption:** Encrypt data using algorithms that convert plaintext into unreadable formats (ciphertext) without the decryption key.
- **Code Obfuscation:** Used in software development to make source code difficult to read or reverse-engineer by renaming variables, removing comments, or altering the control flow.
- **Masking and Shuffling:** Alters data in a database so that it is not easily identifiable while maintaining its usability for testing or analysis.

Tools Used: Common tools include **Dotfuscator** (for .NET applications), **ProGuard** (for Java applications), **JavaScript Obfuscator**, and **ConfuserEx**.

3. Timestamping: This technique is used to modify the timestamps of files to hide the actual timeline of events. Digital files have metadata that records when they were created, accessed, or modified. By manipulating these timestamps, someone can mislead Forensic Investigators about the actual dates and times of these actions.

Common Methods

- **File System Timestamps:** Use tools to alter the metadata stored in file systems to change the recorded timestamps.
- **Obfuscation:** Timestamp alteration can confuse the event's timeline, making it challenging for Forensic Investigators to determine when a particular activity happened.

Tools Used: Some common tools include **Touch**, **Timestamp**, and various **PowerShell scripts**.

4. Clearing Event Logs: This technique is used to delete or modify system logs that track the activity of user and system events. These logs are crucial for forensic investigations as they provide a historical record of activities taken on a system.

Common Methods

- **Manual Deletion:** Manually access and delete log files stored by operating systems or applications.
- **Log Alteration Tools:** Utilize specialized software to modify or erase certain entries in logs to remove or hide traces of particular activities.
- **Scripted Cleansing:** Automated scripts can be written to clear or alter logs regularly.
- **Log Fabrication:** Generate fake logs to mislead Forensic Investigators, making a false narrative of system activity.

Tools Used: Tools used include **ClearLogs**, **Metasploit Framework**, **PowerShell,** and **Wipe**.

5. File Deletion: This technique is used to remove files from a system to prevent Forensic Investigators from accessing them. However, simple deletion does not entirely erase data; it can be recovered until overwritten.

Common Methods

- **Standard Deletion:** Operating systems mark the storage space as free without physically removing the data. This data can often be recovered unless overwritten.
- **Secure File Deletion:** Overwrites data with random bytes multiple times to make it unrecoverable.

Tools Used: Popular tools include **Eraser**, **BleachBit**, **Secure Remove**, and **SDelete**.

6. Disk Wiping: This technique is used to completely erase the data from a storage device (like a hard drive or SSD,) making it almost impossible to recover the original data. It overwrites the actual data stored on the disk.

Common Methods

- **Overwriting Techniques:** Overwrites the disk's storage space with random data, zeros, or a specific pattern multiple times.
- **Secure Deletion:** Overwritten data in such a way that even advanced forensic techniques like magnetic force microscopy cannot recover it.

Tools Used: Popular tools include **DBAN** (Darik's Boot and Nuke), **Eraser**, and **CCleaner**.

7. Program Packers: This technique is used to compress or encrypt executable files to hide their contents and make analysis more complex. It is often used to protect malicious software (like malware) from being easily detected by antivirus programs.

Common Methods

- **Compression:** Reduce the size of the executable file by embedding a decompression routine that restores the original executable at runtime.
- **Encryption:** The executable code is encrypted, requiring decryption at runtime, making static analysis by forensic experts more complex.
- **Anti-Reversing Techniques:** Include additional anti-debugging features that detect and prevent reverse engineering.

Tools Used: Common tools include **UPX (Ultimate Packer for Executables)**, **VMProtect** (Virtual Machine Protect), and **PECompact** (Portable Executable Compact).

8. Data Carving: This technique recovers deleted or hidden data from unallocated disk space, slack space, or memory dumps without relying on the file system structure.

Common Methods

- **Extracting Hidden Data:** Attackers retrieve sensitive information from disk sectors not indexed by the file system, often using file signatures to identify data fragments.
- **Hiding Evidence:** Attackers may place sensitive data in unallocated space, making it difficult for standard forensic tools to detect.

Tools Used: Some common tools include **Scalpel**, **Foremost**, and **TestDisk.**

PRACTICALS using KALI LINUX

Steganography : Steghide Tool

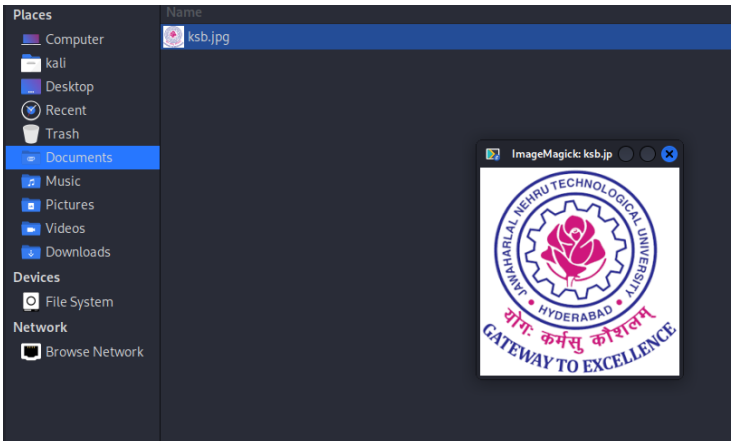
- Sudo apt update
- Sudo apt upgrade
- Sudo apt install steghide

```
File  Actions  Edit  View  Help
┌──(kali㉿kali)-[~]
└─$ sudo apt install steghide
[sudo] password for kali:
steghide is already the newest version (0.5.1-15).
The following packages were automatically installed and are no longer required:
  amass-common              libdaxctl1                libgumbo2           librdmacm1t64                       python3-kismetcapturertladsb
  firebird3.0-common        libegl-dev                libhdf4-0-alt       libre2-10                           python3-kismetcapturertlamr
  firebird3.0-common-doc    libfmt9                   libhdf5-103-1t64    libsframe1                          python3-mistune0
  firmware-ti-connectivity  libfreerdp-client2-2t64   libhdf5-hl-100t64   libsigsegv2                         python3-packaging-whl
  fonts-liberation2         libfreerdp2-2t64          libibverbs1         libsnmp40t64                        python3-pendulum
  freerdp2-x11              libgail-common            libicu-dev          libsuperlu6                         python3-protobuf
  gir1.2-girepository-2.0   libgail18t64              libiniparser1       libtagc0                            python3-pysmi
  hydra-gtk                 libgav1-1                 libinstpatch-1.0-2  libu2f-udev                         python3-pytzdata
  ibus-gtk                  libgdal34t64              libjim0.82t64       libutempter0                        python3-setuptools-scm
  ibverbs-providers         libgeos3.12.1t64          libjs-jquery-ui     libvdpau-va-gl1                     python3-trove-classifiers
  icu-devtools              libgfapi0                 libjsoncpp25        libwinpr2-2t64                      python3-wheel-whl
  libabsl20230802           libgfrpc0                 liblab-gamut1       libzip4t64                          python3-zombie-imp
  libann0                   libgfxdr0                 liblbfgsb0          mesa-vdpau-drivers                  python3.11
```

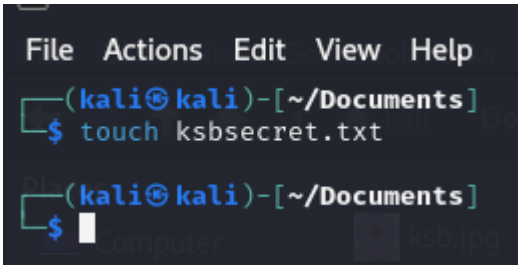
- $ steghide –help (embed & extract)
  - Embedding options -ef : select the file to embed
  - -cf: select cover file
  - -p : passphrase : password to protect hidden content
  - Extracting optins -sf: --stegofile, image containing the hidden data

```
 File  Actions  Edit  View  Help
 help, --help            display this usage information

embedding options:
 -ef, --embedfile        select file to be embedded
   -ef <filename>        embed the file <filename>
 -cf, --coverfile        select cover-file
   -cf <filename>        embed into the file <filename>
 -p, --passphrase        specify passphrase
   -p <passphrase>       use <passphrase> to embed data
 -sf, --stegofile        select stego file
   -sf <filename>        write result to <filename> instead of cover-file
 -e, --encryption        select encryption parameters
   -e <a>[<m>]|<m>[<a>]  specify an encryption algorithm and/or mode
   -e none               do not encrypt data before embedding
 -z, --compress          compress data before embedding (default)
   -z <l>                 using level <l> (1 best speed ... 9 best compressi
 -Z, --dontcompress      do not compress data before embedding
 -K, --nochecksum        do not embed crc32 checksum of embedded data
 -N, --dontembedname     do not embed the name of the original file
 -f, --force             overwrite existing files
 -q, --quiet             suppress information messages
 -v, --verbose           display detailed information

extracting options:
 -sf, --stegofile        select stego file
   -sf <filename>        extract data from <filename>
 -p, --passphrase        specify passphrase
   -p <passphrase>       use <passphrase> to extract data
 -xf, --extractfile      select file name for extracted data
   -xf <filename>        write the extracted data to <filename>
 -f, --force             overwrite existing files
 -q, --quiet             suppress information messages
 -v, --verbose           display detailed information

options for the info command:
 -p, --passphrase        specify passphrase
   -p <passphrase>       use <passphrase> to get info about embedded data

To embed emb.txt in cvr.jpg: steghide embed -cf cvr.jpg -ef emb.txt
To extract embedded data from stg.jpg: steghide extract -sf stg.jpg
```

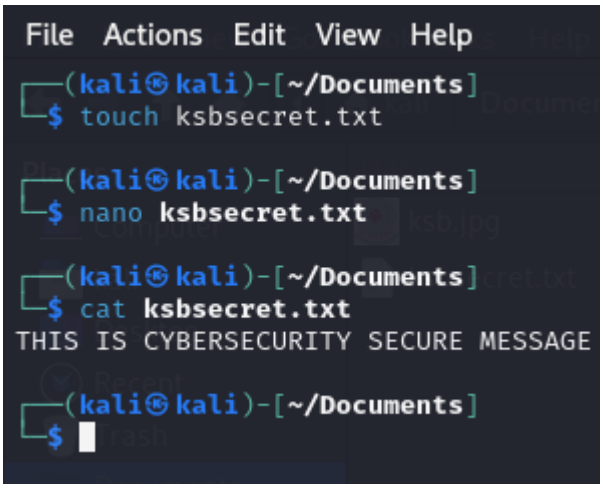
- Create a cover image **ksb.jpg** : save it in the folder



- From the same folder, open the terminal and create a text file : touch ksbsecret.txt



- Open the secret.txt file : write a message .
  $ nano ksbsecret.txt
  THIS IS CYBERSECURITY SECURE MESSAGE



- Save the file
- **EMBEDDING PROCESS:** Now, Embed the secret.file into cover file cover.jpg using steghide tool
  - $ steghide embed - - embedfile ksbsecret.txt - -coverfile ksb.jpg
  - Enter passphrase: CYBER, REENTER: CYBER

```
┌──(kali㉿kali)-[~/Documents]
└─$ steghide embed --embedfile ksbsecret.txt --coverfile ksb.jpg
Enter passphrase:
Re-Enter passphrase:
embedding "ksbsecret.txt" in "ksb.jpg"... done
```

- Once the embedded process is completed delete secret.txt file from the folder
- **EXTRACTION PROCESS** : In extraction we have only the cover image ksb.jpg. To extract
  - **steghide extract** - - stegofile ksb.jpg
  - **Enter passphrase**: CYBER

```
┌──(kali㉿kali)-[~/Documents]
└─$ steghide extract --stegofile ksb.jpg
Enter passphrase:
wrote extracted data to "ksbsecret.txt".
```

  - **The extracted data is written to secret**.txt (check the folder for the file)

DELETING LOGS

- To see history
  $ history
- To clear history
  $ history - c
- Once again see history (All History Deleted)
  $ history
- To know the name of history file
  $ echo $HISTFILE
  /home/kare/.bash_history
- To remove history file
  $ rm .bash_history
- TO REMOVE SYSTEM LOGS
- System logs are stored in var logs
  - $ cd /var/log
  - $ ls
- To remove a log file
  - $ sudo rm auth.log
  - $ sudo rm syslog
- Instead of removing a file log, we can make file empty
  - Sudo cat kern.log
  - Sudo truncate -s 0 kern.log
  - Sudo cat kern.log
- To delete all files in a directory
  - Sudo rm -rf /tmp/*
- **With rm, files may not be deleted permanently, for that we use secure delete tools like WIPE**
- Install secure delete(wipe)
  - $ sudo apt install secure-delete
  - To delete the file

        - $ sudo shred -vfzu xyz.java
- To remove an installed tool
    - Sudo apt remove - - purge secure-delete
- Finally Reboot the SYSTEM
    - $ sudo reboot